COMP 2233: DATA STRUCTURES

Module h3-1: Programming Languages, C Structure & Development Pipeline

Assoc. Prof. Dr. Ömer ÇETİN

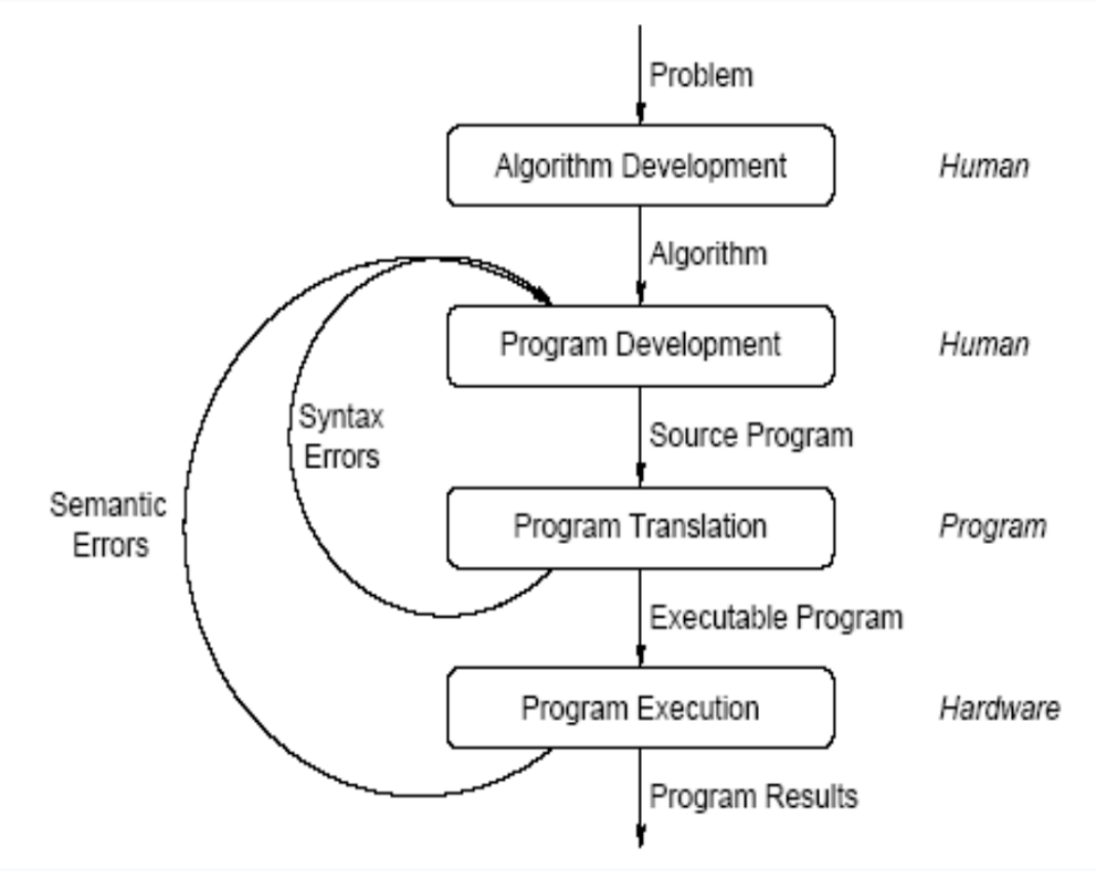
Problem Solving Cycle & Hardware Execution

Human & Software Layers

- Problem Definition: Identifying real-world computational requirements.
- Algorithm Development: Human logic designing step-by-step solutions.
- Program Development: Translating algorithms into C source code.
- Program Translation: Software compilers converting code to executable binary.

Hardware Execution & Error Types

- Program Execution: Physical CPU hardware running binary machine instructions.
- Program Outputs: Generating final computational results.
- Syntax Errors: Code rule violations caught during compilation/translation loop.
- Semantic Errors: Logical flaws caught during program execution testing.

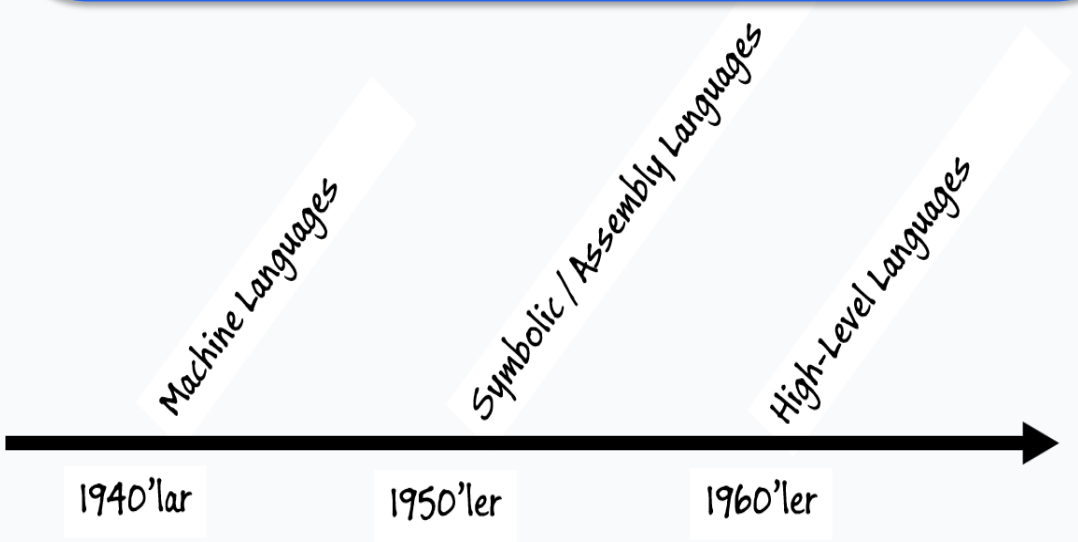


LANGUAGE EVOLUTION

Evolution Timeline of Programming Languages

Chronological Development

- **1940s: Machine Languages** consisting strictly of 0s and 1s.
- **1950s: Symbolic / Assembly Languages** using mnemonic codes (e.g., MOV, ADD).
- **1960s+: High-Level Languages** approaching natural human language (Fortran, C, Pascal).



Key Evolutionary Trends

- **Hardware Abstraction**: Moving from physical circuit details to abstract problem logic.
- **Programmer Productivity**: Dramatic reduction in coding time and syntax complexity.
- **Portability**: Code becoming independent of specific computer architecture.

Machine & Assembly Languages

LOW-LEVEL LANGUAGES

Machine Language (First Generation)

- Binary Operations: Instructions expressed solely in 0s and 1s.
- Hardware Directness: Direct control over CPU registers and memory addresses.
- High Complexity: Extremely difficult for humans to read, write, and debug.

Assembly Language (Second Generation)

- Symbolic Mnemonics: Uses short text codes (e.g., `SUB`, `MUL`, `MOV`).
- Assembler Translation: Requires an Assembler program to convert to binary.
- Machine Dependency: Tied directly to specific processor instruction sets.

Multiplication Program in Machine Language

Binary Instruction Structure

- Raw Binary Stream: Every line contains memory addresses, opcodes, and operands.
- No Readability: Humans cannot discern logic without instruction reference manuals.
- Error Prone: A single flipped bit causes catastrophic program crashes.
- 1940s Hardware Execution: Programmers toggled switches or fed punch cards.

```
// Machine Code Sample (Binary)

00000000 00000100 0000000000000000
01011110 00001100 11000010 0000000000000010
         11101111 00010110 0000000000000101
         11101111 10011110 0000000000001011
11111000 10101101 11011111 0000000000010010
         01100010 11011111 00000000000010101
11101111 00000010 11111011 0000000000010111
11101000 10101101 11011111 0000000000011110
```

Characteristics of High-Level Languages

Human-Centric Design

- Natural Syntax: Uses English words, mathematical symbols, and logical expressions.
- Abstract Hardware: Hides register management, memory offsets, and CPU quirks.
- Faster Development: Programmers focus on problem logic rather than CPU cycles.

Translation Requirement

- Compiler / Interpreter: High-level code must be translated into machine binary.
- Error Diagnostics: Modern compilers identify syntax errors with precise line numbers.
- Examples: C, C++, Java, Python, Pascal, Fortran.

Multiplication Program in Symbolic (Assembly) Language

Assembly Mnemonic Structure

- Mnemonics: Uses commands like `entry`, `subl2`, `pushal`, `calls`, `mull3`.
- Register References: Explicitly references CPU registers (`r2`, `sp`, `fp`).
- Better Than Binary: More readable than 0s and 1s, but still tied to machine architecture.

```
// Assembly Language Sample
entry main, ^m<r2>
subl2 #12, sp
jsb   C$MAIN_ARGS
movab $CHAR_STRING_CON
pushal -8(fp)
pushal (r2)
calls #2, SCANF
pushal -12(fp)
pushal 3(r2)
calls #2, SCANF
mull3 -8(fp), -12(fp), r0
pusha 6(r2)
calls #2, PRINTF
clrl  r0
ret
```

Multiplication Program in C Language

High-Level C Structure

- Structured & Clear: English keywords (`int`, `main`, `scanf`, `printf`).
- Variable Names: Descriptive identifiers (`number1`, `number2`, `result`).
- Hardware Independence: Same C code runs on Intel, ARM, or RISC processor after recompilation.

// Complete C Code Sample

```c
/* Program reads two integers and prints product */
#include <stdio.h>

int main(void) {
    int number1, number2, result;

    printf("Enter two numbers: ");
    scanf("%d %d", &number1, &number2);

    result = number1 * number2;
    printf("Product: %d\n", result);
    return 0;
}
```

LANGUAGE CLASSIFICATION

Hierarchy Pyramid of Programming Languages

Classification Tiers

- Very High-Level (4GL): VB.NET, SQL, Access (tells computer WHAT to do).
- High-Level: Pascal, BASIC, Fortran (abstracts hardware).
- Middle-Level: C, C++, C#, Java, Ada (combines high-level constructs with direct memory pointers).
- Low-Level & Machine: Assembly & Binary (0s and 1s).

Why C is Called Middle-Level

- High-Level Constructs: Structured control flow, functions, user-defined types.
- Low-Level Access: Direct memory pointers, bitwise manipulation, hardware address access.
- Optimal Power: Offers high performance needed for systems programming.

Core Reasons to Learn C Language

Algorithmic Foundation & Portability

- Algorithmic Thinking: Excellent language for mastering programming logic and data structures.
- Hardware Independence: Portable C code compiles across virtually all hardware platforms.
- Rich Ecosystem: Abundant compiler tools, IDEs, and standard libraries available worldwide.

Structured Paradigm & Industry Influence

- Structured Programming: Modular design based on functions and clear scope boundaries.
- Foundation of Modern Languages: Inspired C++, Java, C#, JavaScript, PHP, Python.
- Real-World Projects: Powers production systems beyond simple textbook exercises.

Real-World Practical Applications of C

System Software & Embedded Systems

- **Device Drivers:** Direct hardware communication buffers.
- **Operating Systems:** UNIX Kernel, Windows core components, Android system layers.
- **Embedded Systems:** Automotive ECUs, IoT devices, robotics controllers.

Language Runtimes & Computation

- **Language Runtimes:** Python C-CPython interpreter, PHP core, Perl, Node.js V8 components.
- **Computational Engines:** MATLAB, Mathematica, high-performance scientific simulations.
- **Game Engines:** Graphics drivers, 3D render pipelines, physics solvers.

DEVELOPMENT PIPELINE

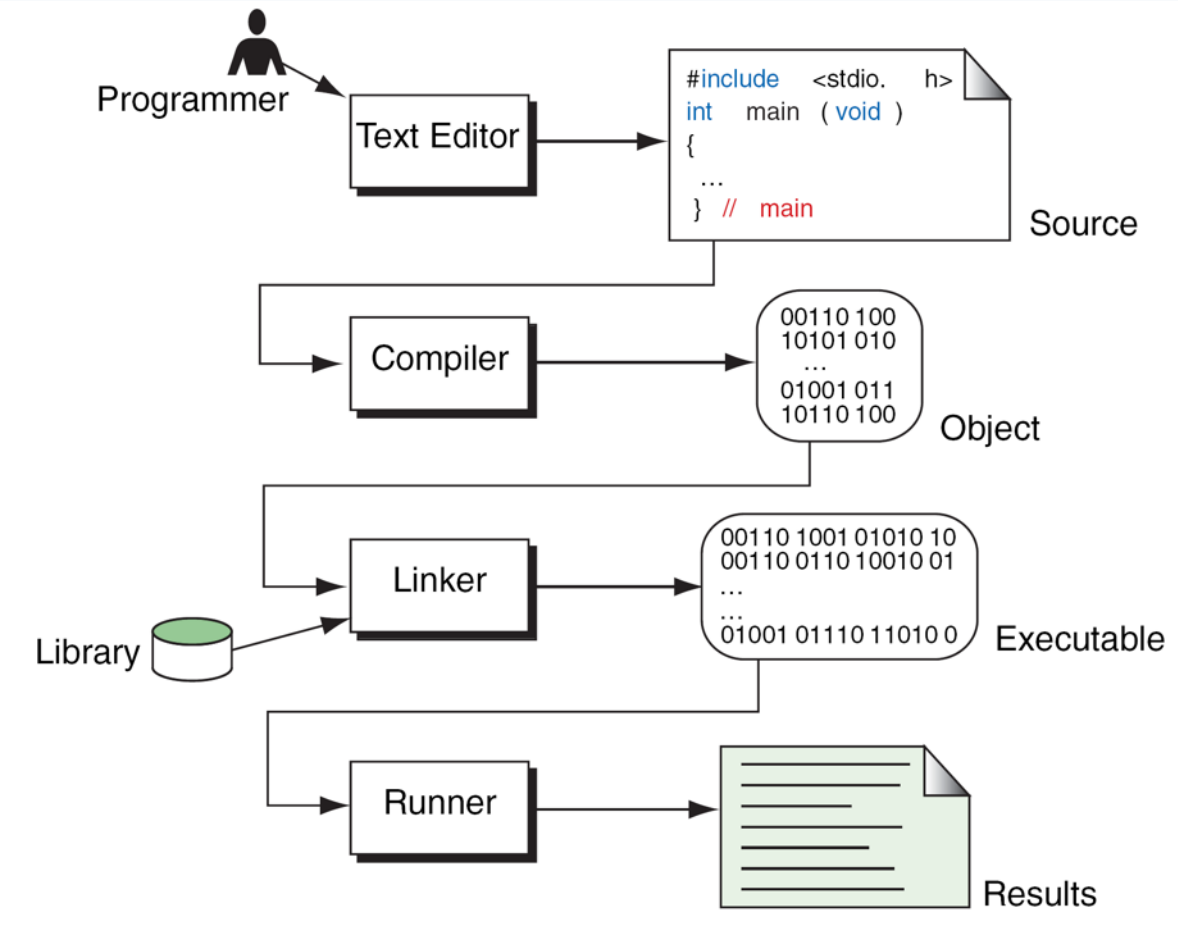
C Compilation & Execution Pipeline

Four Pipeline Stages

- 1. Text Editor: Writing human-readable source code file (`main.c`).
- 2. Compiler: Translating source code into object binary code (`main.o` / `main.obj`).
- 3. Linker: Combining object code with C Standard Libraries (`stdio.h`) into executable (`main.exe`).
- 4. Runner / Execution: CPU executing binary program and printing outputs.

Iterative Error Feedback Loops

- Syntax Error Loop: Caught during Compilation -> Return to Text Editor to fix typos.
- Semantic / Runtime Loop: Caught during Execution -> Return to Algorithm Design or Code.



LEARNING OBJECTIVES

Objectives of C Programming Foundations

Key Language Pillars

To understand the structure of the C programming language;

- To write our first program in C,
- To introduce preprocessor commands,
- To create good identifiers for objects in a program,
- To list, explain and use basic C data types,
- To create and use variables and constants,
- To understand the concepts of input and output,
- To use simple input and output statements…

C is a **structured programming** language. It is considered a high-level language because it allows the programmer to concentrate on the problem at hand and not worry about the machine (in the hardware sense) that the program will use. This is another reason why it is used by software developers whose applications need to run on many different hardware platforms.

Comments in C Programming Language

Comment Types & Purpose

- Compiler Ignore: Comments are ignored by the compiler and produce 0 machine instructions.
- Multi-line Comments `/* ... */`: Block comments spanning multiple lines.
- Single-line Comments `// ...`: Line comments running to the end of the line.
- Best Practice: Use comments to explain non-obvious algorithm logic and function purpose.

```
// Comment Usage Examples
/*
 * Multi-line Comment Block
 * Program: Hello World Example
 * Author: Assoc.Prof.Dr. Ömer ÇETİN
 */

#include <stdio.h>

int main(void) {
    int a = 5; // Inline single-line comment
    printf("Hello World\n");
    return 0;
}
```

DEVELOPMENT TOOLS

C Compilers & Execution Environments

Online Compilers (Quick Testing)

- OnlineGDB (www.onlinegdb.com): Browser-based C/C++ compiler and debugger.
- Zero Installation: Ideal for quick syntax verification without local setup.
- Console Terminal: Provides interactive input/output execution windows.

Professional Local IDEs

- VSCode: Modern lightweight code editor configured with GCC/G++ GDB toolchain.
- CLion / Visual Studio: Full-featured IDEs with advanced memory inspection and profilers.
- Code::Blocks / Dev-C++: Traditional desktop C development environments.

Minimal Valid C Program Structure

Minimal Program Components

- Valid Execution: Smallest possible C program that compiles cleanly without errors.
- `int main()`: Mandatory entry-point function required by C standard.
- Braces `{ ... }`: Defines the execution body block.
- `return 0;`: Signals successful execution exit status to the operating system.

```c
// Minimal C Code
int main(void) {
    return 0;
}
```

General Structure of a C Program

General Structural Sections

- **1. Header Documentation Comments:** Program author, date, and description.
- **2. Preprocessor Includes:** Adding Standard I/O headers (`#include <stdio.h>`).
- **3. Main Function Entry:** `int main(void)`.
- **4. Statements & Functions:** Executable logic (`printf`, `return 0`).

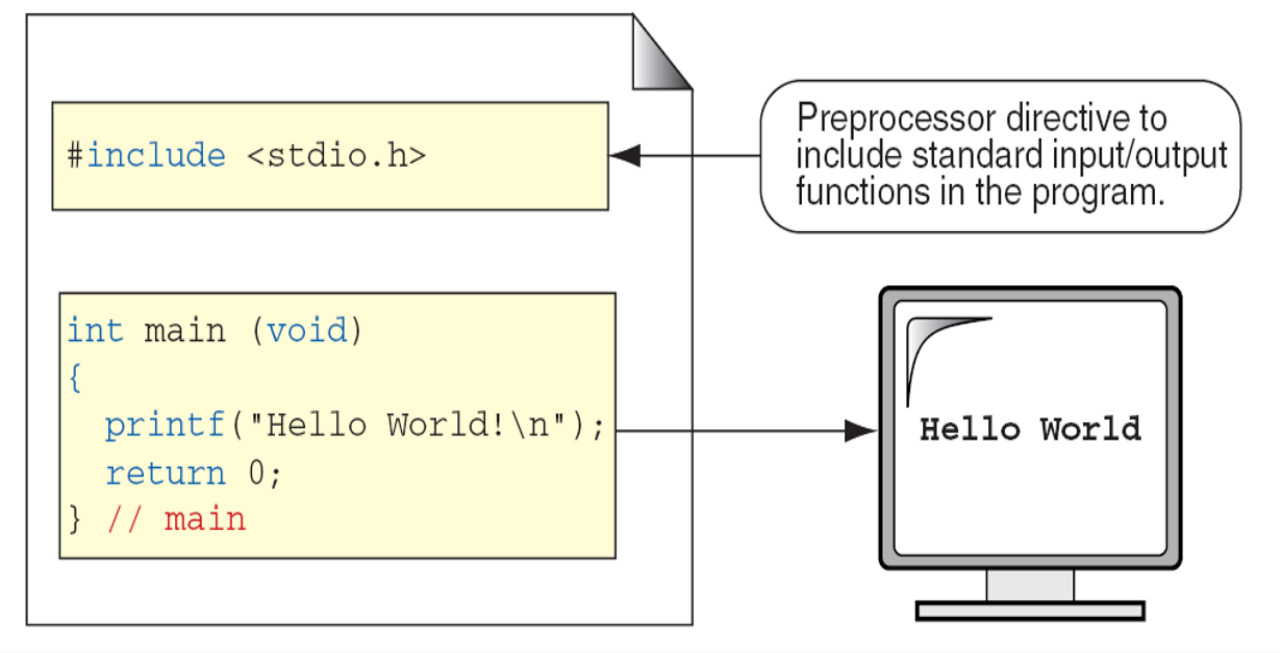
```
// General Template Code

/*
 * Program Name: First C Program
 * Description: Standard Hello World
 */

#include <stdio.h>

int main(void) {
    // Execution Block
    printf("Merhaba Dunya\n");
    return 0;
}
```

Preprocessor Directives & Standard Libraries



Preprocessor Mechanism (`#`)

- **Hash Symbol (`#`):** Lines beginning with `#` are processed before compilation starts.
- **Header Inclusion:** `#include <stdio.h>` inserts the Standard I/O function declarations.
- **Standard Library Capabilities:** Provides access to `printf()`, `scanf()`, `getchar()`, `putchar()`.

Why Standard Headers Matter

- **No Reinventing Wheel:** Programmers don't write low-level screen printing routines.
- **Type Checking:** Ensures arguments passed to `printf()` match expected library signatures.
- **Portability:** Standard headers are uniform across Windows, Linux, and macOS.

printf() Function & Format Specifiers

STANDARD OUTPUT

printf() Function Mechanics

- Standard Output: Writes formatted text stream to console terminal.
- Format Identifiers: `%s` (string), `%d` (integer), `%f` (float), `%c` (char).
- Semicolon Terminal: Every statement MUST end with `;` to indicate termination.
- Multi-Argument Scanning: Evaluates identifiers left-to-right matching comma arguments.

// Multi-Argument printf Example

```
#include <stdio.h>

int main(void) {
    int books = 2, compilers = 3;
    char lang[] = "C Programming";

    printf("Learning %s with %d books and %d compilers.\n",
           lang, books, compilers);
    return 0;
}
```

MODULE SUMMARY

Summary & Takeaways of Module h3-1

Core Concepts Learned

- Language Evolution: Transition from Machine Binary to Structured C.
- Development Pipeline: Editing -> Compilation -> Linking -> Hardware Execution.
- C Features: Hardware independence, middle-level power, systems applicability.

Syntax Essentials

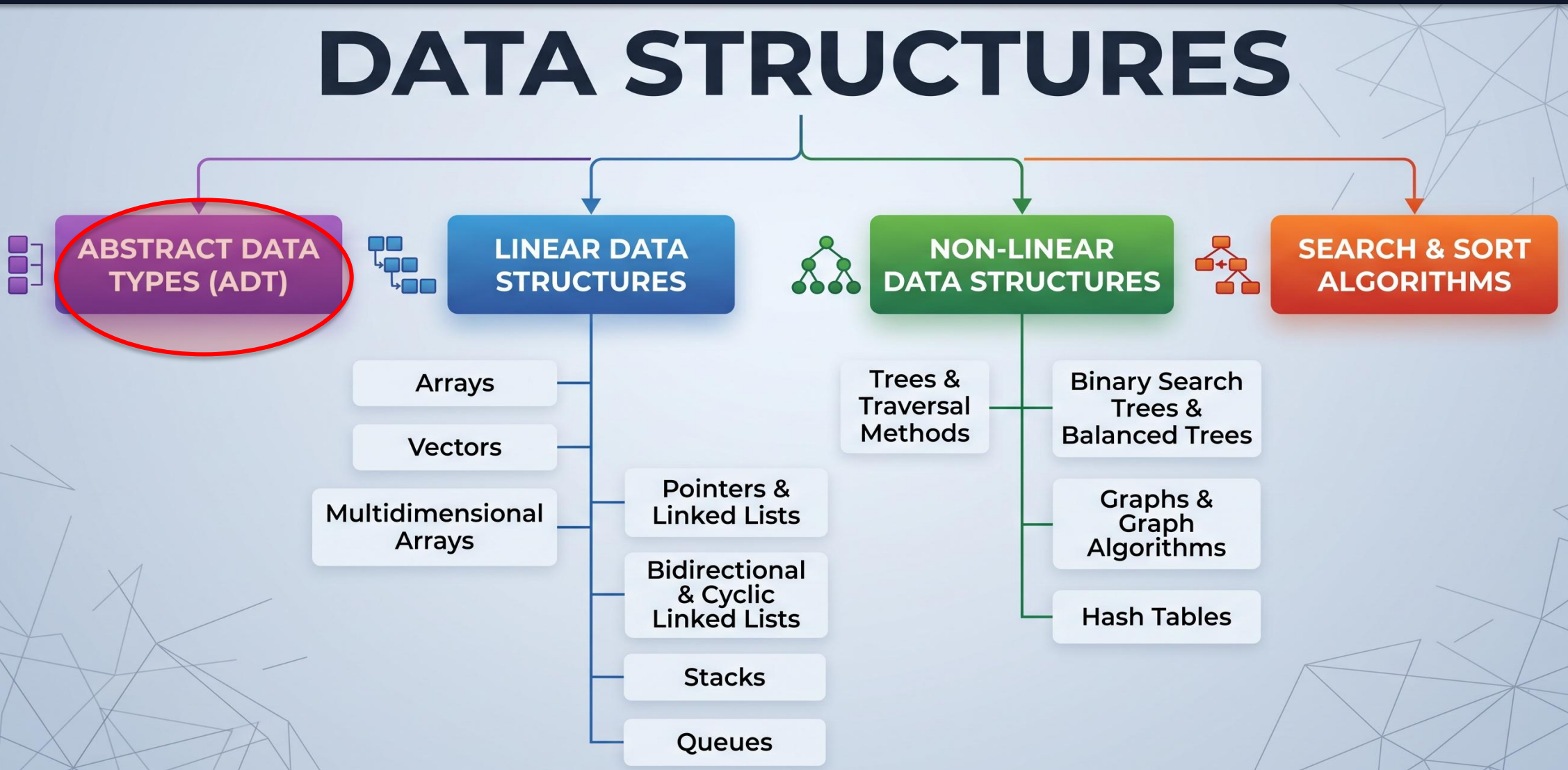
- Program Skeleton: `#include <stdio.h>`, `int main()`, `{ ... }`, `return 0;`.
- Statements & Semicolons: Semicolons `;` mark statement boundaries.
- Console Output: Formatted printing via `printf()` and specifiers (`%d`, `%s`).

COMP 2233: DATA STRUCTURES

Module h3-2: Identifiers, Variables, Scope, Data Types & Input Processing

Assoc. Prof. Dr. Ömer ÇETİN

Course Content



Identifiers & Memory Naming in C

Identifier Concept

- Naming Mechanism: Identifiers are textual names assigned to program objects (variables, functions, arrays).
- Unique Memory Addresses: Every identifier is linked to a unique physical RAM memory location.
- Programmer Abstraction: Enables referencing memory via readable names (e.g., `id1`) rather than raw hex addresses (`0x7FFF54`).

Identifier Declaration Examples

- `int id1, id2;`: Declares two integer identifiers occupying 4 bytes each in RAM.
- `char str[20];`: Declares a character array identifier allocating 20 contiguous bytes.
- `double total_amount;`: Declares a floating-point identifier allocating 8 bytes.

Variables & RAM Memory Allocation

Variable Characteristics

- Named Storage Cells: Basic memory units that store data values during program execution.
- Value Mutation: Contents stored inside a variable can be modified (mutated) dynamically.
- Type Requirement: Every C variable MUST be declared with an explicit data type before use.
- Address Mapping: Variable names act as symbolic aliases to underlying RAM addresses.

Why Declaration is Mandatory

- Memory Reservation: Tells compiler how many bytes of RAM to reserve (1 byte for `char`, 4 for `int`).
- Operations Enforcement: Defines allowed mathematical and logical operations.
- Type Safety: Prevents illegal bit operations on incompatible memory representations.

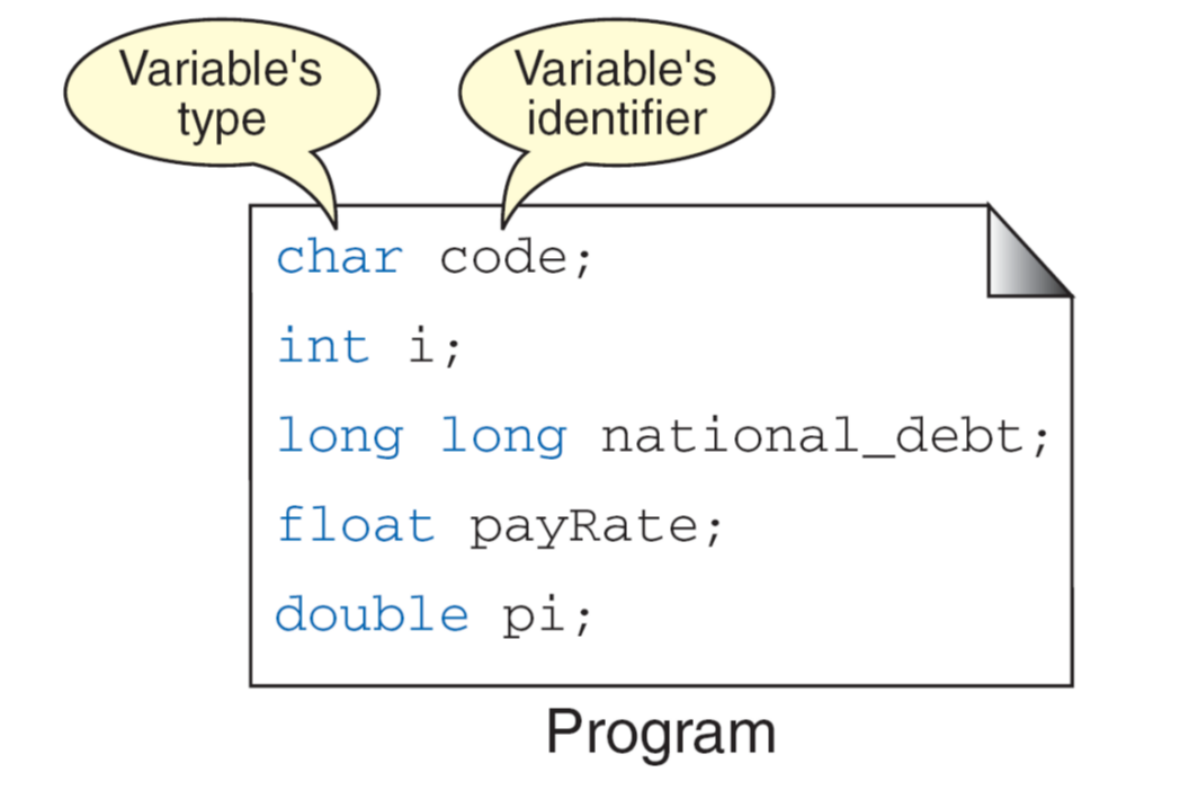
Strict C Variable Naming Rules

Permissible Syntax Rules

- First Character: MUST begin with an alphabetic letter (`a`-`z`, `A`-`Z`) or an underscore `_`.
- Subsequent Characters: Can contain letters, digits (`0`-`9`), and underscores `_`.
- Case Sensitivity: C is strictly case-sensitive! `Number`, `number`, and `NUMBER` are 3 separate variables.
- Underscore Allowed: `total_score` and `_tempValue` are valid.

Forbidden Syntax Violations

- No Leading Digits: `1stNumber` is ILLEGAL (must be `number1`).
- No Special Characters: Spaces, `!`, `?`, `@`, `-`, `{`, `]` are strictly FORBIDDEN.
- No Reserved Keywords: Cannot use C keywords (`int`, `for`, `if`, `return`, `while`).
- No Non-ASCII Characters: Avoid non-English characters (`ç`, `ğ`, `ş`, `ı`, `ö`, `ü`).



Declaration of Variables in C

Declaration Patterns

- Single Variable Declaration: `DataType IdentifierName;`.
- Multiple Variable Declaration: `DataType id1, id2, id3;` (allocates 3 variables of same type).
- Equivalence: `int a, b, c;` is functionally identical to 3 separate statements (`int a; int b; int c;`).
- Memory Allocation: Happens immediately when function execution enters variable scope.

```
// Declaration Code Samples
#include <stdio.h>

int main(void) {
    // Single declarations
    int count;
    float average;

    // Multiple declarations in one line
    int x, y, z;
    double gpa, total_score;

    return 0;
}
```

Uninitialized Variables & Garbage Values

The Garbage Value Threat

- No Automatic Zeroing: Declaring a local variable does NOT automatically set its value to zero!
- Garbage Memory: The variable contains whatever leftover bit patterns previously existed in that RAM location.
- Undefined Behavior: Reading an uninitialized variable leads to unpredictable program results and bugs.

Initialization Best Practices

- Explicit Assignment at Declaration: `int sum = 0;` initializes variable upon allocation.
- Read-Before-Use Rule: Always assign a value (via `=` or `scanf`) BEFORE reading the variable in calculations.
- Static/Global Exception: Global variables are automatically zero-initialized by the C runtime.

Memory Value Mutation Trace in C

MEMORY MUTATION

Step-by-Step State Trace

- Line 1: `int a, b, result;` -> Memory allocated (garbage values).
- Line 2: `a = 5; b = 2;` -> Writes 5 into address `a`, 2 into address `b`.
- Line 3: `a = a + 1;` -> Reads 5, adds 1, overwrites address `a` with 6.
- Line 4: `result = a - b;` -> Computes `6 - 2 = 4`, writes 4 into address `result`.
- Output: `printf` displays 4 on screen terminal.

```c
// Memory Mutation Code
#include <stdio.h>

int main(void) {
    int a, b, result;

    a = 5;
    b = 2;
    a = a + 1;      // a becomes 6
    result = a - b; // 6 - 2 = 4

    printf("Result: %d\n", result); // 4
    return 0;
}
```

Global vs. Local Variable Scope in C

Global Variables (Program-Wide)

- Placement: Declared outside all function blocks (top of file).
- Scope: Accessible and mutable from ANY function across the entire program.
- Lifetime: Persists in RAM for the entire execution duration of the application.

Local Variables (Block-Restricted)

- Placement: Declared inside function or curly brace block `{ ... }`.
- Scope: Strictly restricted to the function or block in which it is defined.
- Lifetime: Allocated on call stack upon block entry, destroyed upon block exit.
- Isolation: Local variables in different functions with same name are completely independent!

SCOPE DEMO

Global & Local Scope Code Demonstration

Scope Execution Flow

- Global Counter: `int globalVar = 10;` modified inside `myFunction()` to 20.
- Global Persistence: Changes made to global variables inside functions persist after function return!
- Local Isolation: `int localVar = 15;` inside `main()` is distinct from `localVar = 5;` inside `myFunction()`.
- Independent Stack Frames: Identical local variable names in separate functions occupy separate RAM addresses.

```
// Scope C++ / C Code Sample
#include <stdio.h>

int globalVar = 10; // Global

void updateGlobal(void) {
    int localVar = 5; // Local to updateGlobal
    globalVar = 20;   // Mutates global
}

int main(void) {
    int localVar = 15; // Local to main
    updateGlobal();
    printf("Global: %d | Local: %d\n", globalVar, localVar);
// 20, 15
    return 0;
}
```

What is a Data Type & Memory Width?

Data Type Role

- Memory Representation: Defines the specific bit pattern layout for storing data in RAM.
- Byte Allocation: Determines exact number of bytes allocated (1 byte = 8 bits).
- Permissible Operations: Specifies valid mathematical/logical operations (e.g., bitwise shifts vs floating divide).

Binary Memory Foundation

- Bit (Binary Digit): Fundamental unit of storage (0 or 1).
- Byte Grouping: 8 bits = 1 byte (represents 256 unique states, `0` to `255`).
- Hardware Alignment: CPUs read RAM in word boundaries (32-bit / 64-bit blocks).

TYPE TAXONOMY

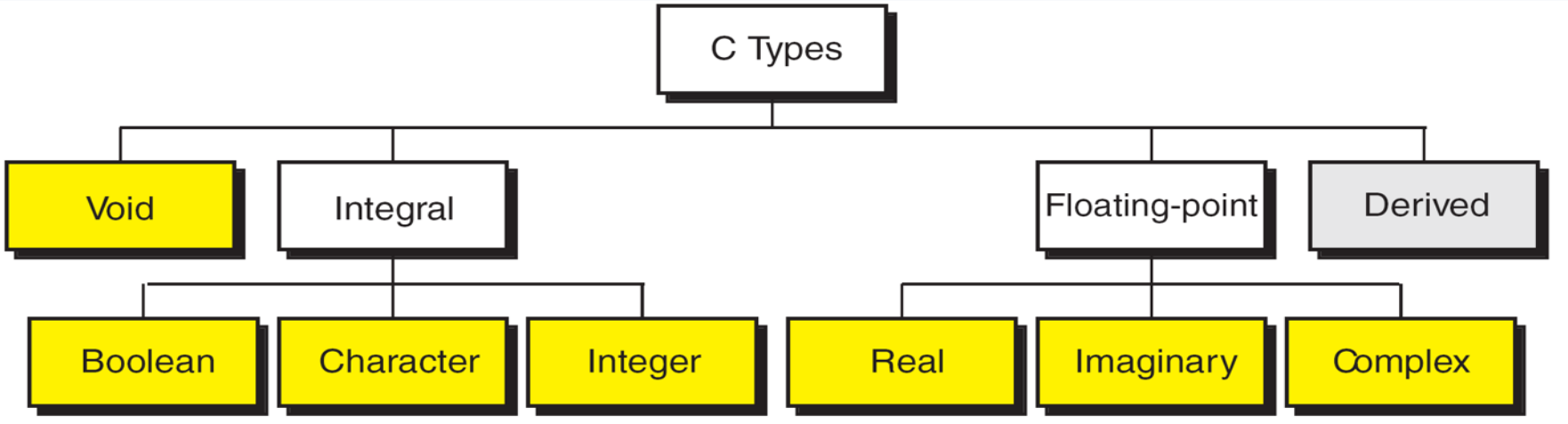
Hierarchy Classification of C Data Types

Primary Category Classification

- Void Type (`void`): Empty type indicating no value or empty function parameters.
- Integral Types: Whole number types (Boolean `bool`, Character `char`, Integer `int`).
- Floating-Point Types: Real decimal numbers (`float`, `double`, `long double`).
- Derived & User-Defined: Composite structures (`arrays`, `pointers`, `struct`, `union`).

Fundamental Primitive Types

- `char`: 1 Byte (-128 to 127 or 0 to 255). Format: `%c`.
- `int`: 2 or 4 Bytes (-2,147,483,648 to 2,147,483,647). Format: `%d`.
- `float`: 4 Bytes single-precision floating point. Format: `%f`.
- `double`: 8 Bytes double-precision floating point. Format: `%lf`.



C Type Modifiers & Size Hierarchy

Type Modifiers

- `signed`: Allows positive and negative values (default for `int`).
- `unsigned`: Non-negative values only (doubles positive range!). E.g., `unsigned int` (0 to 4,294,967,295).
- `short`: Reduces memory size (e.g., `short int` = 2 bytes).
- `long`: Expands memory size (e.g., `long long int` = 8 bytes / 64 bits).

Size Inequality Constraint

- C Standard Size Rule: `sizeof(float) <= sizeof(double) <= sizeof(long double)`.
- `sizeof()` Operator: Returns physical byte width of any type or variable at compile time.
- Platform Precision: Ensures floating-point precision scales predictably across systems.

Type	Byte Size	Minimum Value	Maximum Value
short int	2	–32,768	32,767
int	4	–2,147,483,648	2,147,483,647
long int	4	–2,147,483,648	2,147,483,647
long long int	8	–9,223,372,036,854,775,807	9,223,372,036,854,775,806

sizeof (short) ≤ sizeof (int) ≤ sizeof (long) ≤ sizeof (long long)

L-Value vs. R-Value Assignment Mechanics

Assignment Semantics Rules

- L-Value (Left Hand Side): Must represent a writable memory location (e.g., variable `a`).
- R-Value (Right Hand Side): An expression, literal, or variable that evaluates to a value.
- Chained Assignment: `a = b = c = 5;` evaluates right-to-left (assigns 5 to `c`, then `b`, then `a`).
- Expression Assignment: `b = 2 + (a = 5);` sets `a = 5` first, then `b = 7`.

```
// Assignment Code Example
#include <stdio.h>

int main(void) {
    int a = 8, b, c;

    b = 2 + (a = 5); // a=5, b=7
    printf("a=%d, b=%d\n", a, b);

    a = b = c = -100; // Chained assignment
    printf("a=%d, b=%d, c=%d\n", a, b, c);
    return 0;
}
```

String Variables & Null Terminator (`\0`)

C String Storage Architecture

- No Native String Type: C has no primitive string type; strings are stored as `char` arrays.
- Null Terminator (`\0`): Essential boundary byte marking the end of text content in memory.
- Buffer Allocation Rule: A string holding N characters requires array size of at least `N + 1` bytes!
- Double Quotes: String literals are enclosed in double quotes `"Hello World"`.

```
// String Declaration Sample
#include <stdio.h>

int main(void) {
    // Allocates 20 bytes; terminated by '\0'
    char str1[20] = "Merhaba Dunya";
    char str2[20] = "Nasilsin";

    printf("%s %s\n", str1, str2);
    return 0;
}
```

ESCAPE CHARACTERS

Special Escape Sequences in C Strings

Essential Escape Sequences

- `\n`: Newline (moves cursor to beginning of next line).
- `\t`: Horizontal Tab (advances cursor to next tab stop).
- `\\`: Backslash character (escapes literal `\`).
- `\"`: Double Quote character (embeds quotes inside string literal).

Additional Formatting Characters

- `\r`: Carriage Return (returns cursor to line beginning).
- `\b`: Backspace (deletes preceding character).
- `\a`: Alert / Bell (triggers system audio chime).
- `\'`: Single Quote character.

STANDARD INPUT

Reading Input via scanf() Function

scanf() Function Role

- Standard Input Stream: Reads formatted input data from keyboard buffer.
- Syntax: `scanf("%d", &variable_name);`.
- Format Specifiers: Matches expected input type (`%d` for int, `%f` for float, `%s` for string).
- Execution Wait: Blocks execution until user types input and hits ENTER key.

Format Specifiers for scanf()

- `%d`: Reads integer values into `int` variable address.
- `%f`: Reads single-precision float into `float` variable address.
- `%lf`: Reads double-precision float into `double` variable address.
- `%s`: Reads string characters into array address (note: NO `&` needed for string array!).

Why Address-Of Operator (`&`) is Required

Address-Of (`&`) Mechanics

- Pass-by-Value Limitation: Passing `id` directly passes a copy of its value to `scanf()`.
- Pass-by-Address Requirement: `scanf()` must mutate the original memory cell in `main()`'s stack frame!
- Address Operator (`&id`): Passes the exact RAM memory address of `id` to `scanf()`.
- String Exception: Array names (`char str[20]`) already decay into memory addresses, so no `&` is used for `%s`!

```c
// scanf Code Example with &
#include <stdio.h>

int main(void) {
    int student_id;
    float gpa;

    printf("Enter ID and GPA: ");
    // Pass RAM addresses using &
    scanf("%d %f", &student_id, &gpa);

    printf("ID: %d | GPA: %.2f\n", student_id, gpa);
    return 0;
}
```

Constants & Immutable Values in C

Constant Characteristics

- Immutable Memory: Data values that CANNOT be changed during program execution.
- Type Rigidity: Constants possess explicit data types just like variables.
- Compile-Time Determination: Numerical values are known precisely at compile time.
- Prevent Mistakes: Protects critical parameters (e.g., `PI`, `MAX_USERS`) from accidental modification.

Categories of Constants

- Literal Constants: Raw values in code (e.g., `100`, `3.14159`, `'A'`, `"Text"`).
- Named `const` Variables: Read-only variables declared using `const` keyword.
- Preprocessor `#define`: Text-substitution macro constants.

Defining Constants: `const` Keyword vs. `#define`

Two Constant Paradigms

- 1. `const` Keyword: Declares a read-only variable with strict type checking (`const double PI = 3.14159;`).
- 2. `#define` Preprocessor: Performs macro text replacement prior to compilation (`#define PI 3.14159`).
- Compiler Enforcement: Attempting to assign `PI = 3.14;` to a `const` variable triggers a compile error!
- Modern Recommendation: Prefer `const` variables for type safety and debugger symbol visibility.

```
// Constant Definition Code
#include <stdio.h>
#define MAX_BUFFER 1024 // Preprocessor macro

int main(void) {
    const double PI = 3.14159; // Type-safe const

    // PI = 3.14; // COMPILE ERROR!
    printf("PI: %.5f | Max: %d\n", PI, MAX_BUFFER);
    return 0;
}
```

Type Cast

Type Cast

It is used to convert a variable of one type to another.

In some cases, if you need to, you can temporarily change the data type of a variable.

You can use the type cast feature for this. The general structure of type cast is as follows:

(veri-türü) değişken;

```
int i;
float f = 3.14;
i = (int) f;
      OR
i = int( f );
```

```c
#include <stdio.h>

int main(void)
{
  double dd;

  dd = 632.524;

  printf("%f %d", dd, (int) dd);

  return 0;
}
```

MODULE SUMMARY

Summary & Takeaways of Module h3-2

Core Concepts Learned

- Identifiers & Variables: Memory location naming, declaration syntax, and strict naming rules.
- Scope Architecture: Global variables (persistent) vs. Local variables (stack isolated).
- Primitive Types: Bit/byte representation across `char`, `int`, `float`, and `double`.

I/O & Constants

- Strings & Null Terminator: Character arrays requiring `\0` termination byte.
- Input Processing: `scanf()` requiring the address-of operator (`&`) for scalar variables.
- Immutable Constants: Safety through `const` keywords and `#define` macros.

COMP 2233: DATA STRUCTURES

Module h3-3: Data, Data Structures & Mathematical Data Models

Assoc. Prof. Dr. Ömer ÇETİN

1. What is Data? (Raw Facts vs Information)

Raw Data Concept

- Raw Facts & Figures: Unprocessed values or symbols without inherent context.
- Computer Representation: How real-world events are recorded and stored digitally.
- Example: `12345` (A standalone numeric value with unknown meaning).

Information Transformation

- Contextualized Information: Data processed and structured to carry meaningful context.
- Example: `Ali Yılmaz, ID: 12345, Computer Engineering`.
- Data Forms in Computing: Numerical, textual, logical, audio, and graphical data streams.

2. What is a Data Structure?

Memory Organization & Order

- Physical Memory Arrangement: Defines how data items are organized and stored in RAM.
- Access Efficiency: Designed to maximize retrieval speed and minimize CPU cycles.
- Algorithmic Necessity: Effective algorithms require structured, organized input data.

Everyday & Language Analogy

- Primitive Variables: Even a simple scalar variable is a basic 1-element data structure.
- Warehouse Analogy: Placing items in organized labeled shelves vs dumping in a pile.
- Compound Structures: Combining primitives into structured composite types (`struct`, `union`).

2.1. Basic / Simple Data Structures

Arrays (Contiguous Memory)

- Definition: Stores multiple homogeneous elements in consecutive RAM memory locations.
- Example: `int numbers[5] = {1, 2, 3, 4, 5};`.
- Key Property: $O(1)$ constant-time random access via numeric index calculation.

Pointers & User-Defined Structures (`struct`)

- Pointers: Hold physical memory addresses (`int* ptr = &val;`), enabling indirect access.
- Structures (`struct`): Groups heterogeneous data types under a single composite name.
- Example: `struct Student { string name; int age; };`.

2.2. Advanced Data Structures: Linear Chains

Linked Lists (Node Chains)

- Structure: Sequential nodes containing data payload and a pointer to next node.
- Advantage: Dynamic capacity growth and $O(1)$ fast insertion/deletion at pointers.
- Trade-off: Requires $O(N)$ sequential traversal for search.

Stacks (LIFO) & Queues (FIFO)

- Stack (Last In, First Out): Elements added and removed from the same top end.
- Stack Example: Browser back button history, function call execution stack.
- Queue (First In, First Out): Elements added at rear, removed from front.
- Queue Example: Printer document queue, OS process scheduling.

2.2. Advanced Data Structures: Non-Linear & Hash

Trees & Graphs

- Trees: Hierarchical branching starting from a root node into parent-child nodes.
- Tree Example: OS File system directory tree, Binary Search Tree (BST).
- Graphs: Network of vertices (nodes) interconnected by directional or weighted edges.
- Graph Example: Social network connections, GPS map navigation routing.

Hash Tables (Key-Value Lookups)

- Structure: Maps key identifiers to array slots via a mathematical hash function.
- Advantage: Unmatched $O(1)$ average-time data retrieval.
- Example: Phonebook directory (contact name -> phone number mapping).

3. What is a Data Model?

Conceptual & Mathematical Framework

- Abstract Model: Defines how data is conceptually organized, related, and accessed in a system.
- Relational Status: Establishes mathematical connections among data entities.
- Problem Solving Blueprint: Engineering problems are solved by approximating standard data models.

Data Structure vs Data Model Distinction

- Data Model: The conceptual/mathematical blueprint (e.g., Relational Table Model).
- Data Structure: The physical C/C++ memory implementation in RAM (e.g., Struct Array or B-Tree).

3.1. Hierarchical & 3.2. Network Data Models

Hierarchical Data Model (3.1)

- Tree-Like Structure: Inverted tree taxonomy with strict parent-child relationships.
- One-to-Many: Each child node has exactly one parent node.
- Real-World Example: University Organization (University -> Faculty -> Department -> Courses).

Network Data Model (3.2)

- Graph Network: Flexible structure where records can have multiple parent nodes.
- Many-to-Many: Models complex interconnected real-world relationships.
- Real-World Example: Supply chain systems (A single product supplied by multiple vendors).

3.3. Relational Data Model (RDBMS)

Tabular Relation Architecture

- Tables & Tuples: Stores data inside two-dimensional tables (relations) made of rows and columns.
- Primary Keys: Unique identifiers ensuring row entity integrity.
- Foreign Keys: Establishes relational links across different database tables.
- Industry Standard: Powers SQL databases (PostgreSQL, MySQL, Oracle, SQLite).

Relational Table Example

- Students Table: `[StudentID (PK), Name, DepartmentID (FK)]`.
- Departments Table: `[DepartmentID (PK), DeptName, Faculty]`.
- Relational Query: JOIN operation linking Student records to Department details.

3.4. Object-Oriented & 3.5. Document Models

Object-Oriented Data Model (3.4)

- Encapsulated Objects: Groups data attributes and executable methods into unified Objects/Classes.
- Inheritance & Polymorphism: Supports object-oriented software engineering concepts natively.
- Use Case: Applications building persistent domain objects directly into databases.

Document-Oriented Data Model (3.5)

- Semi-Structured Documents: Stores records as self-contained JSON or XML documents.
- Schema Flexibility: No rigid table definitions; fields vary dynamically per document.
- Industry Example: NoSQL document databases like MongoDB and CouchDB.

SYSTEM IMPORTANCE

Importance of Data Structures & Data Models

Performance & Memory Management

- Runtime Speed: Optimal structure dramatically enhances data retrieval and processing speed.
- Memory Conservation: Compact layouts save RAM and prevent spatial fragmentation.
- Fast Access: Direct indexing and hash functions enable instantaneous search capabilities.

Flexibility & Long-Term Maintenance

- Dynamic Adaptability: Flexible structures adapt smoothly as dataset sizes scale.
- Clean Architecture: Standard data models simplify long-term software refactoring and maintenance.
- Scalability: Allows enterprise systems to handle millions of concurrent user requests.

SELECTION CRITERIA

Choosing the Right Data Structure or Model

Key Evaluation Metrics

- Dataset Size & Growth: Is data static (fixed Array) or dynamic (Vector / Linked List)?
- Access Pattern: Is random $O(1)$ indexing needed or sequential stream traversal?
- Operation Frequencies: Does the system perform more reads, insertions, or deletions?

Constraints & System Goals

- RAM Memory Limits: Hardware memory limits dictate allocation strategies.
- Target Performance: Critical SLAs requiring $O(1)$ vs $O(\log N)$ or $O(N)$ bounds.
- Application Paradigm: RDBMS SQL vs NoSQL JSON document store requirements.

COMPARATIVE MATRIX

Data Structure Selection Matrix

Static vs Dynamic Structures

- Static Array: $O(1)$ access, fixed size, $O(N)$ insertion/deletion.
- Dynamic Vector: $O(1)$ access, dynamic growth, amortized $O(1)$ append.
- Linked List: $O(N)$ access, dynamic growth, $O(1)$ fast node insertion at pointers.

Specialized & Non-Linear Structures

- Stack / Queue: Restricted LIFO/FIFO order for process and buffer management.
- BST Tree: $O(\log N)$ fast ordered search, insertion, and deletion.
- Hash Table: $O(1)$ average time direct key-value lookup.

CONCLUSION

Conclusion & Core Software Engineering Synergy

The Fundamental Synergy Formula

- Niklaus Wirth's Formula: `Programs = Algorithms + Data Structures`.
- Data Core: Data is the central asset processed by computer software.
- Organization Impact: Excellent algorithms fail if fed with unorganized data structures.

Final Takeaways

- Right Tool for Job: Match operational requirements to appropriate data structures.
- Performance Mastery: Data structures directly dictate Big-O time and space complexity.
- Foundation for Future Weeks: Sets the stage for deep-dives into Arrays, Lists, Trees, and Graphs.

MODULE REVIEW

Week 3 Module Overview & Next Steps

What We Covered in Source h3-3

- Data vs Information: Transforming raw figures into contextualized meaning.
- Data Structure Taxonomy: Simple (Arrays, Structs) vs Advanced (Lists, Trees, Hash).
- Data Models: Hierarchical, Network, Relational, Object, and Document Models.
- Selection Criteria: Performance, RAM limits, access frequency.

Upcoming Week 3 Deep-Dives

- Single-Dimension Arrays in C: Memory indexing, operations, C syntax.
- Dynamic Vectors in C++: STL `std::vector`, dynamic resizing, `push_back` mechanics.